

LQCD Master

完整技术文档

Agentic Scientific Computing for Lattice QCD

2026 年 7 月 20 日

项目地址：

https://github.com/sjtu-sai-agents/LQCD_Master.git

许可证：MIT

Python 版本要求：3.10+

目录

1 项目概述	3
1.1 核心能力	3
1.2 流水线架构	4
2 安装指南	4
2.1 环境要求	4
2.2 安装步骤	4
2.3 GPU 后端安装 (可选)	5
3 配置指南	5
3.1 环境配置 (.env)	5
3.2 集群与系综配置 (configs/)	6
3.2.1 config.yaml —— Slurm 与运行时配置	6
3.2.2 ensemble_presets.yaml —— 系综预设	6
3.2.3 skills.yaml —— 技能路由	7
4 命令行接口	7
4.1 基本用法	7
4.2 运行示例	7
5 项目结构	8
6 核心模块详解	9
6.1 WorkflowOrchestrator (工作流编排器)	9
6.2 PlannerAgent (规划代理)	10
6.3 ExecutorAgent (执行代理)	10
6.4 技能系统	11
7 基准测试	11
7.1 验证任务体系	11
7.2 实验评估结果	12
8 详细使用示例	12
8.1 示例 1: 计算 Pion 两点关联函数	12
8.2 示例 2: 计算质子两点关联函数	13
8.3 示例 3: 计算 Wilson 圈	13
8.4 示例 4: 从文件读取任务	13
8.5 示例 5: 非交互式全流程	13

9 接口清单 (API Reference)	13
9.1 run.py ——入口点	13
9.2 WorkflowOrchestrator	14
9.3 PlannerAgent	15
9.4 ExecutorAgent	15
9.5 技能系统	16
9.6 配置系统	17
9.7 Prompt 系统	17
10 常见使用模式	18
10.1 交互式探索	18
10.2 批量测试	18
10.3 自定义技能扩展	18
11 依赖关系	19
12 贡献指南	19

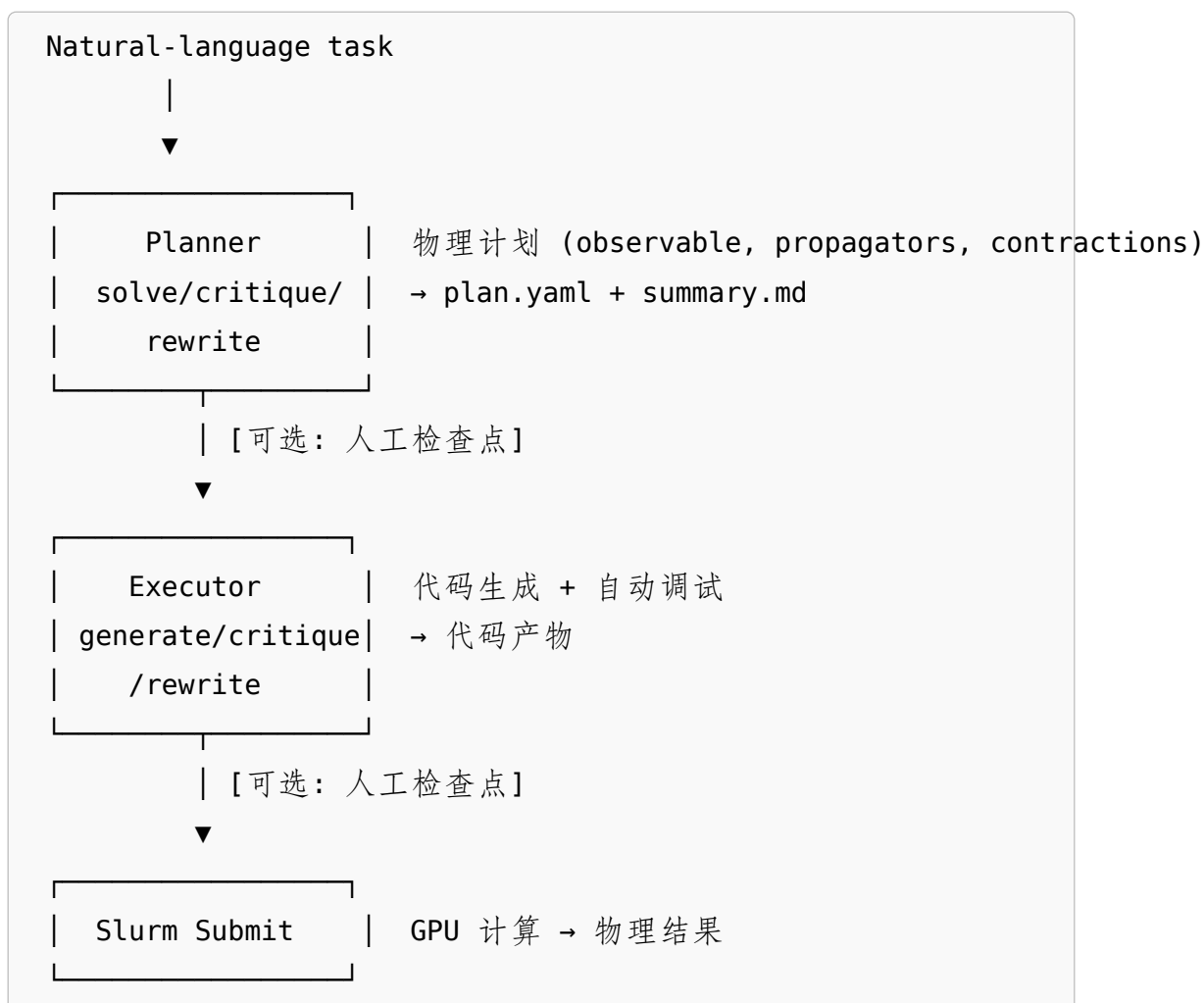
1 项目概述

LQCD Master 是一个自主 **AI 科学家**系统，专为格点量子色动力学 (Lattice QCD) 研究设计。它将自然语言描述的物理任务自动翻译为可执行的 PyQUDA 程序及 Slurm 集群提交脚本。

1.1 核心能力

- **自然语言到 GPU 计算**: 用户用自然语言描述物理目标 (如” 计算 pion 两点关联函数”), 系统自动生成完整的 PyQUDA 代码
- **多阶段流水线**: Planner (物理规划) → Executor (代码生成 + 自动调试) → Slurm 提交
- **人在回路**: 每个阶段都设有检查点 (checkpoint), 集群提交前必须经人工确认
- **自动调试**: 静态分析检测到代码错误后, Executor 自动重写修正
- **技能路由**: 领域知识 (关联函数理论、PyQUDA API、分析方法等) 按任务相关性注入 LLM 提示词
- **基准验证**: 70 个独立验证任务, 覆盖 5 类可观测量

1.2 流水线架构



2 安装指南

2.1 环境要求

2.2 安装步骤

```

1 # 1. 克隆仓库
2 git clone https://github.com/sjtu-sai-agents/LQCD_Master.git
3 cd LQCD_Master
4
5 # 2. 安装 Python 依赖
6 pip install -r requirements.txt
7
8 # 3. 配置环境变量
9 cp .env.template .env
10 # 编辑 .env 填入 LLM API Key 和 Serper Key

```

组件	要求
Python	≥ 3.10
GPU 后端	CuPy (CUDA 12.x 或 ROCm 5.x)
格点 QCD	PyQUDA + QUDA (从源码安装)
集群调度	Slurm with GPU partition
LLM API	OpenAI 兼容端点 (如 DeepSeek)
Web 搜索	Serper.dev API Key

Python 依赖	
openai	≥ 1.0
prompt_toolkit	≥ 3.0
PyYAML	≥ 6.0
numpy	≥ 1.24
opt_einsum	≥ 3.3

表 1: 系统需求

```
11
12 # 4. 运行任务
13 python run.py --task "Calculate the pion two-point correlator" --test --non-
    interactive
```

2.3 GPU 后端安装 (可选)

```
1 # CUDA 12.x
2 pip install cupy-cuda12x
3
4 # ROCm 5.x
5 pip install cupy-rocm-5-0
```

3 配置指南

3.1 环境配置 (.env)

```
1 # LLM API 配置
2 OPENAI_API_KEY=sk-your-api-key-here
3 OPENAI_BASE_URL=https://api.deepseek.com/v1
4 OPENAI_MODEL=deepseek-v4-pro
5
```

```

6 # Web 搜索工具 (Serper.dev)
7 SERPER_API_KEY=your-serper-api-key-here

```

3.2 集群与系综配置 (configs/)

3.2.1 config.yaml —— Slurm 与运行时配置

```

1 workflow:
2   executor_static_check_rounds: 1      # 静态检查最大轮次
3   executor_dynamic_test_rounds: 1     # 动态测试最大轮次
4
5 script:
6   shell: /bin/bash
7
8 slurm:
9   partition: kshdexplu16
10  nodes: 1
11  gres: dcu:4
12  exclude: f17r3n00
13  ntasks_per_node: 4
14  ntasks_per_socket: 1
15  exclusive: true
16
17 ensemble:
18   preset: C24P29                      # 系综预设名称
19   cfg_num: ["10000"]                 # 组态编号
20
21 runtime:
22   module_purge: true
23   modules:
24     - compiler/gnu/9.3.0
25     - mpi/hpcx/2.11.0/intel-2017.5.239
26     - compiler/dtk/25.04.4
27   exports:
28     QUDA_PATH: /path/to/quda
29     CUPY_INSTALL_USE_HIP: "1"
30   activate: source /path/to/venv/bin/activate
31   launch_mpi: mpirun
32   launch_python: python3

```

3.2.2 ensemble_presets.yaml —— 系综预设

定义 7 个格点系综的参数，包含规范组态路径、夸克质量、Clover 系数和多网格参数。

3.2.3 skills.yaml ——技能路由

控制每个流水线阶段注入哪些领域知识模块。五个内置技能：

- **关联函数理论**：两点/三点函数的谱分解、基态提取方法
- **能谱分析**：有效质量、拟合窗口选择、激发态处理
- **PyQUDA API**：场创建、Dirac 算子、求解器、HMC 用法
- **规范固定**：库仑规范、Landau 规范
- **数据分析**：Jackknife、Bootstrap、误差传播

4 命令行接口

4.1 基本用法

```
1 python run.py [OPTIONS]
```

参数	说明
--task "<text>"	内联任务描述（跳过交互式提示）
--task <file>	从文件读取任务描述
--test	仅生成代码，跳过 Slurm 提交
--run-dir <path>	输出目录（默认: runs/<timestamp>）
--non-interactive	自动接受所有检查点
--dotenv-path <path>	自定义.env 文件路径
--list-skills	列出可用技能并退出

表 2: CLI 参数

4.2 运行示例

```
1 # 交互式运行
2 python run.py
3
4 # 非交互式，测试模式
5 python run.py --task "Calculate pion 2pt function with momentum Pz=2" \
6     --test --non-interactive
7
8 # 列出可用技能
9 python run.py --list-skills
```



```

10
11 # 指定输出目录
12 python run.py --task "Compute proton two-point correlator" \
13     --run-dir my_experiment

```

5 项目结构

```

1 LQCD_Master/
2 |— run.py                # CLI 入口点
3 |— requirements.txt      # Python 依赖
4 |— .env.template        # 环境变量模板
5 |— core_architecture/   # 核心流水线
6 |   |— orchestrator.py  # 工作流编排
7 |   |— planner.py       # Planner 代理
8 |   |— executor.py      # Executor 代理
9 |— utils/               # 工具模块
10 |   |— llm_client.py    # LLM 客户端
11 |   |— tool_client.py   # 内置工具客户端
12 |   |— submit_tool.py   # Slurm 提交工具
13 |   |— prompt_loader.py # 提示词加载器
14 |   |— skill_utils.py   # 技能系统 (Registry/Router/Runner)
15 |   |— config_utils.py  # 配置工具
16 |   |— io_utils.py      # I/O 工具
17 |   |— static_check.py  # 静态代码分析
18 |   |— generate_einsum/  # Wick 收缩与 einsum 代码生成
19 |— prompts/            # 阶段提示词模板
20 |   |— planner/         # solve / critique / rewrite
21 |   |— executor/       # generate / critique / rewrite
22 |— configs/            # 集群、系综、技能配置
23 |— skills/             # 领域知识
24 |— benchmark/          # 70 个验证任务
25 |   |— 2pt_local/       # 20 个局域两点函数任务
26 |   |— 2pt_nonlocal/    # 10 个非局域两点函数任务
27 |   |— wilson_loop/     # 12 个 Wilson 圈任务
28 |   |— 3pt_meson/       # 13 个介子三点函数任务
29 |   |— 3pt_baryon/      # 15 个重子三点函数任务
30 |— experiments/        # 实验与模型评估结果

```

6 核心模块详解

6.1 WorkflowOrchestrator（工作流编排器）

core_architecture/orchestrator.py 中的 WorkflowOrchestrator 类是整个系统的中央控制器。

```
1 class WorkflowOrchestrator:
2     def __init__(
3         self,
4         *,
5         task: str,                # 用户任务描述
6         run_dir: Path,            # 运行目录
7         llm_client: Any,          # LLM 客户端
8         tool_client: Any,         # 工具客户端
9         submit_tool: Any,         # Slurm 提交工具
10        test_mode: bool = False,  # 测试模式
11        non_interactive: bool = False, # 非交互模式
12        max_revision_rounds: int = 3, # 最大修订轮次
13    ): ...
14
15    def run(self) -> dict[str, Any]:
16        """执行完整流水线，返回最终状态字典"""
17
18    def _run_planner_stage(self) -> tuple[dict, int]:
19        """运行 Planner 阶段（含人工检查点循环）"""
20
21    def _run_executor_stage(self, planner_output) -> tuple[dict, int]:
22        """运行 Executor 阶段（含静态检查与自动重写循环）"""
23
24    def _run_test_submission_stage(self, bundle, version, planner_output):
25        """运行测试提交流程（含 Slurm 队列监控）"""
```

执行流程：

1. Planner 生成物理计划 → 人工检查点确认
2. Executor 生成代码包 → 静态分析 → 自动修复循环
3. 人工检查点确认后提交测试作业
4. 监控 Slurm 队列状态与输出
5. 若有错误则自动重写并重新提交

6.2 PlannerAgent (规划代理)

core_architecture/planner.py 实现了三阶段物理规划流程：

```

1 class PlannerAgent:
2     def __init__(self, llm_client, tool_client, run_dir): ...
3
4     def run(self, task: str) -> dict[str, Any]:
5         """Solve → Critique → Rewrite 完整流程"""
6
7     def rewrite_with_feedback(self, task, current, feedback, version):
8         """基于人工反馈进行计划修订"""
9
10    # 三阶段
11    # 1. solve: 根据任务生成初始物理计划
12    # 2. critique: 评审计划的可行性和完整性
13    # 3. rewrite: 根据评审反馈修订计划

```

Planner 输出格式 (plan.yaml) 包含：

- task: ——任务描述
- ensemble: ——系综参数 (格点大小、间距、夸克质量等)
- physics: ——物理设置 (强子类型、动量、规范固定)
- measurement: ——测量方法 (关联函数构造、收缩方案)
- output: ——输出文件与格式
- extras: ——额外参数

6.3 ExecutorAgent (执行代理)

core_architecture/executor.py 实现了代码生成、静态分析、自动修复的完整流程：

```

1 class ExecutorAgent:
2     def __init__(self, llm_client, submit_tool, run_dir,
3                 tool_client=None, max_static_check_rounds=None): ...
4
5     def generate_bundle(self, *, task, planner_output, version,
6                        feedback=None, previous_bundle=None) -> dict:
7         """生成完整代码包 (主程序 + Slurm 脚本 + 静态分析 + 评判) """
8
9     def submit_test(self, bundle, version) -> dict:
10        """提交测试作业到 Slurm"""
11

```

```
12 def submit_full(self, bundle, version) -> dict:
13     """提交完整作业到 Slurm"""
14
15 def collect_evidence_paths(self, bundle, version) -> list[str]:
16     """收集所有产物路径"""
```

Executor 子阶段:

1. **generate**: 根据 Planner 输出生成 PyQUDA 代码 + Slurm 脚本
2. **static_analysis**: 静态分析检测代码错误
3. **critique**: LLM 评审生成代码的质量和正确性
4. **rewrite**: 根据评审和静态分析结果自动重写
5. 步骤 2-4 循环执行直至无错误或达到最大轮次

6.4 技能系统

技能系统位于 `utils/skill_utils.py`, 提供了可扩展的领域知识注入框架:

```
1 class SkillRegistry:
2     """技能注册表: 从 skills/ 目录加载领域知识模块"""
3     def load(self) -> None
4     def names(self) -> list[str]
5
6 class SkillRouter:
7     """技能路由器: 根据任务选择相关技能"""
8     def route(self, context: SkillContext) -> list[str]
9
10 class SkillRunner:
11     """技能运行器: 执行 LLM 交互循环"""
12     def run(self, context, system_prompt, user_message,
13             fallback_tools=None) -> tuple[dict, list, list]
14
15 class SkillMessageAssembler:
16     """技能消息组装器: 将领域知识注入 LLM 对话"""
```

7 基准测试

7.1 验证任务体系

`benchmark/` 目录定义了 **70 个独立验证任务**, 覆盖 5 类可观测量, 全部基于 C24P29 系综 (cfg 10000, 点源, 零动量)。

可观测量类别	任务数	示例
2pt local（局域两点）	20	$\pi, K, \rho, D, D_s, J/\psi, p, \Lambda, \Xi, \Sigma, \Lambda_c, \Xi_c, \Omega_c$
2pt nonlocal（非局域两点）	10	$\pi, K, \rho, D, D_s, J/\psi$ 的 Wilson 线偏移 $z = 0 \dots 10$
Wilson loop（Wilson 圈）	12	$W(R, T), R, T \in \{1, 2, 3, 4\}$, 平均于 XT, YT, ZT 平面
3pt meson（介子三点）	13	$D \rightarrow K/\pi, B \rightarrow D/K/\pi, D \rightarrow K^*, B \rightarrow K^*, D_s \rightarrow \phi, K \rightarrow \pi,$
3pt baryon（重子三点）	15	$p \rightarrow p, \Lambda \rightarrow \Lambda, \Lambda \rightarrow p, \Lambda_c \rightarrow \Lambda, \Xi_c \rightarrow \Xi, \Lambda_b \rightarrow \Lambda_c/\Lambda/\Sigma$

表 3: 基准任务分类

7.2 实验评估结果

在完整 70 任务的基准套件上进行交叉验证：

实验	骨干模型	精确匹配	符号翻转	准确率
LQCD Master GPT-5.4	GPT-5.4	63	3	90.0%
LQCD Master DeepSeek	DeepSeek V4 Pro	56	2	80.0%

表 4: 实验评估结果

- **精确匹配**：与标准方法在机器精度 ($|\Delta| < 10^{-12}$) 或相对误差 $< 10^{-3}$ 范围内一致
- **符号翻转**：全局 γ_5 -厄米性相位约定，不影响物理结果
- **失败**：真正的代码生成或执行错误

8 详细使用示例

8.1 示例 1：计算 Pion 两点关联函数

```
1 # 交互式运行
2 python run.py
3 # 输入：Calculate the pion two-point correlator on C24P29 ensemble
4 #       with momentum Pz=2 in the z-direction
5
6 # 非交互式测试模式
7 python run.py \
8     --task "Calculate the pion two-point correlation function with \
9           momentum smearing, Pz=2, on C24P29 ensemble cfg 10000" \
10    --test --non-interactive
```

8.2 示例 2: 计算质子两点关联函数

```
1 python run.py \  
2     --task "Compute the proton two-point correlator using distillation \  
3           with momentum smearing, Pz=3, Coulomb gauge fixed" \  
4     --test --non-interactive
```

8.3 示例 3: 计算 Wilson 圈

```
1 python run.py \  
2     --task "Compute Wilson loops W(R,T) for R=1..4, T=1..4 on C24P29, \  
3           average over all spatial planes" \  
4     --test
```

8.4 示例 4: 从文件读取任务

```
1 # 创建任务文件  
2 cat > my_task.txt << 'EOF'  
3 Calculate the D meson to kaon semileptonic three-point function  
4 on C24P29 ensemble, cfg 10000, point source at t0=0,  
5 sequential propagator through the sink at tsink=20,  
6 vector current insertion.  
7 EOF  
8  
9 # 运行任务  
10 python run.py --task my_task.txt --test --non-interactive
```

8.5 示例 5: 非交互式全流程

```
1 python run.py \  
2     --task "Compute kaon two-point correlator with point source, \  
3           Pz=0, on C24P29 cfg 10000" \  
4     --run-dir runs/kaon_2pt \  
5     --non-interactive
```

9 接口清单 (API Reference)

9.1 run.py —— 入口点

```

1 def build_parser() -> argparse.ArgumentParser:
2     """构建命令行参数解析器"""
3
4 def resolve_run_dir(user_run_dir: str) -> Path:
5     """解析运行目录路径"""
6
7 def print_banner() -> None:
8     """打印 LQCD Master ASCII 横幅"""
9
10 def prompt_task(prefilled: str = "") -> str:
11     """交互式任务提示（支持预填充）"""
12
13 def list_skills() -> list[str]:
14     """列出所有可用技能名称"""
15
16 def main() -> None:
17     """CLI 主入口点"""

```

9.2 WorkflowOrchestrator

```

1 class WorkflowOrchestrator:
2     _TERMINAL_SLURM_STATES = {
3         "COMPLETED", "FAILED", "CANCELLED", "TIMEOUT",
4         "OUT_OF_MEMORY", "NODE_FAIL", "PREEMPTED",
5         "BOOT_FAIL", "DEADLINE",
6     }
7
8     def __init__(self, *, task, run_dir, llm_client,
9                 tool_client, submit_tool, test_mode=False,
10                 non_interactive=False, max_revision_rounds=3)
11
12     # 主要方法
13     def run(self) -> dict[str, Any]
14     def _run_planner_stage(self) -> tuple[dict, int]
15     def _run_executor_stage(self, planner_output) -> tuple[dict, int]
16     def _run_test_submission_stage(self, bundle, version, planner_output)
17
18     # 检查点
19     def _checkpoint_planner(self, output, version) -> tuple[bool, str]
20     def _checkpoint_executor(self, bundle, version) -> tuple[bool, str]
21
22     # Slurm 监控
23     def _inspect_slurm_queue(self, bundle, submitted_job_id) -> dict

```

```

24 def _inspect_slurm_job(self, job_id) -> dict
25 def _inspect_sacct_job(self, job_id) -> dict
26 def _wait_for_test_job_output(self, *, bundle, submitted_job_id,
27                               poll_interval_seconds=5,
28                               timeout_seconds=1800) -> dict
29 def _query_test_job_status(self, submitted_job_id) -> dict
30
31 # 辅助方法
32 def _save_state(self) -> None
33 def _build_test_failure_feedback(self, monitor_status) -> str
34 def _test_monitor_has_error(self, monitor_status) -> bool

```

9.3 PlannerAgent

```

1 class PlannerAgent:
2     def __init__(self, llm_client, tool_client, run_dir)
3
4     # 主要方法
5     def run(self, task: str) -> dict[str, Any]
6     def rewrite_with_feedback(self, task, current, feedback, version) -> dict
7     def validate_plan_coverage(self, plan_yaml: str) -> dict[str, Any]
8     def debug_state(self) -> dict[str, Any]
9
10    # 内部方法
11    def _run_stage(self, *, stage, build_messages, task) -> dict
12    def _chat_json_autonomous(self, user_message, *, stage, task) -> tuple
13    def _load_system_prompt(self, stage: str) -> str
14    def _build_stage_user_message(self, *, stage, payload, task) -> str
15    def _to_planner_output(self, payload, fallback=None) -> dict
16    def _normalize_critique_payload(self, payload) -> dict
17    def _load_config(self, path) -> dict
18    def _build_fixed_facts_yaml(self) -> str
19    def _persist_final(self, output) -> None

```

9.4 ExecutorAgent

```

1 class ExecutorAgent:
2     def __init__(self, llm_client, submit_tool, run_dir,
3                 tool_client=None, max_static_check_rounds=None)
4
5     # 主要方法
6     def generate_bundle(self, *, task, planner_output, version,
7                       feedback=None, previous_bundle=None) -> dict

```



```

8     def generate_and_submit_test(self, task, planner_output, version,
9                                   feedback=None, previous_bundle=None) -> tuple
10    def submit_full(self, bundle, version) -> dict
11    def submit_test(self, bundle, version) -> dict
12    def collect_evidence_paths(self, bundle, version) -> list[str]
13
14    # 内部阶段方法
15    def _run_executor_stage(self, *, task, plan_yaml, summary_md,
16                            submit_config_yaml, input_facts_yaml,
17                            executor_dir="") -> tuple
18    def _run_critique_stage(self, *, task, plan_yaml, summary_md,
19                            main_program, test_submit_script,
20                            full_submit_script, static_analysis_report) -> tuple
21    def _run_rewriter_stage(self, *, task, plan_yaml, summary_md,
22                            submit_config_yaml, input_facts_yaml,
23                            executor_dir="", current_code,
24                            critique_summary, errors, warnings,
25                            feedback) -> tuple
26    def _run_static_analysis_stage(self, *, main_program,
27                                   test_submit_script,
28                                   full_submit_script) -> dict
29
30    # 配置与 I/O
31    def _load_submit_config(self, path) -> dict
32    def _collect_input_facts(self) -> dict
33    def _probe_lime_header(self, path) -> dict
34    def _render_submit_script(self, submit_spec, *, script_dir) -> str
35
36    # 验证
37    def _require_payload_field(self, payload, key) -> str
38    def _require_submit_spec(self, payload, key, *, expected_program) -> dict
39    def _require_config_value(self, section, key) -> str
40    def _require_process_grid(self, ensemble_cfg) -> list[int]

```

9.5 技能系统

```

1 class SkillRegistry:
2     def __init__(self, skills_dir: Path)
3     def load(self) -> None
4     def names(self) -> list[str]
5     def get(self, name: str) -> dict
6
7 class SkillRouter:
8     def __init__(self, registry, skills_config_path)

```

```

9     def route(self, context: SkillContext) -> list[str]
10
11 class SkillContext:
12     stage: str
13     task: str
14     payload: dict
15
16 class SkillRunner:
17     def __init__(self, llm_client, registry, router, assembler, tool_registry)
18     def run(self, context, system_prompt, user_message,
19            fallback_tools=None) -> tuple[dict, list, list]
20
21 class SkillMessageAssembler:
22     def assemble(self, skill_names, context) -> str
23
24 class ToolRegistry:
25     def __init__(self, tool_client)
26     def get(self, name: str) -> callable

```

9.6 配置系统

```

1 # config_utils.py
2 def load_yaml_mapping(path: Path) -> dict[str, Any]
3 def resolve_submit_config_presets(data, ensemble_presets) -> dict
4 def build_prompt_ensemble_yaml(raw_config, ensemble_presets) -> str
5 def build_prompt_submit_config_yaml(raw_config, ensemble_presets) -> str
6 def build_runtime_launch(runtime_cfg: dict) -> str
7
8 # static_check.py
9 def analyze_bundle_static(*, main_program, test_submit_script,
10                          full_submit_script) -> dict

```

9.7 Prompt 系统

```

1 class PromptLoader:
2     def __init__(self, prompt_dir: Path)
3     def load(self, relative_path: str) -> str

```

提示词模板组织：

- prompts/planner/solve_system.md ——Planner 求解阶段系统提示
- prompts/planner/solve_user.md ——Planner 求解阶段用户提示模板

- `prompts/planner/critique_system.md` ——Planner 评审阶段系统提示
- `prompts/planner/critique_user.md` ——Planner 评审阶段用户提示模板
- `prompts/planner/rewrite_system.md` ——Planner 修订阶段系统提示
- `prompts/planner/rewrite_user.md` ——Planner 修订阶段用户提示模板
- `prompts/planner/plan_template.yaml` ——计划输出模板
- `prompts/executor/generate_system.md` ——Executor 代码生成系统提示
- `prompts/executor/generate_user.md` ——Executor 代码生成用户提示模板
- `prompts/executor/critique_system.md` ——Executor 代码评审系统提示
- `prompts/executor/critique_user.md` ——Executor 代码评审用户提示模板
- `prompts/executor/rewrite_system.md` ——Executor 代码重写系统提示
- `prompts/executor/rewrite_user.md` ——Executor 代码重写用户提示模板

10 常见使用模式

10.1 交互式探索

```
1 python run.py
2 # 系统会提示输入任务描述
3 # 在 Planner 和 Executor 完成后都会暂停等待人工确认
```

10.2 批量测试

```
1 # 批量运行多个任务
2 for task_file in tasks/*.txt; do
3     python run.py --task "$task_file" --test --non-interactive \
4         --run-dir "runs/$(basename $task_file .txt)"
5 done
```

10.3 自定义技能扩展

在 `skills/` 目录下创建新的 YAML 文件：

```
1 # skills/my_custom_skill.yaml
2 name: my_custom_skill
3 description: "Custom domain knowledge for specific analysis"
4 stages: [planner_solve, executor_generate]
```

```
5 keywords: [keyword1, keyword2]
6 content: |
7   # Custom Domain Knowledge
8   ...detailed knowledge content...
```

11 依赖关系

依赖	版本	用途
Python	≥ 3.10	运行环境
openai	≥ 1.0	LLM API 客户端
prompt_toolkit	≥ 3.0	交互式输入
PyYAML	≥ 6.0	YAML 配置解析
numpy	≥ 1.24	数值计算
opt_einsum	≥ 3.3	优化张量收缩
CuPy	按 CUDA/ROCm	GPU 加速
PyQUDA + QUDA	从源码安装	格点 QCD 计算
Slurm	—	集群作业调度
Serper.dev API	—	Web 搜索（可选）

表 5: 完整依赖列表

12 贡献指南

1. 自定义 configs/config.yaml 和 configs/ensemble_presets.yaml 以适配您的集群
2. 在 skills/ 下添加新技能以扩展领域知识
3. 在 benchmark/ 中扩展验证任务和标准方法参考
4. 对新 LLM 骨干模型运行交叉验证以评估性能